

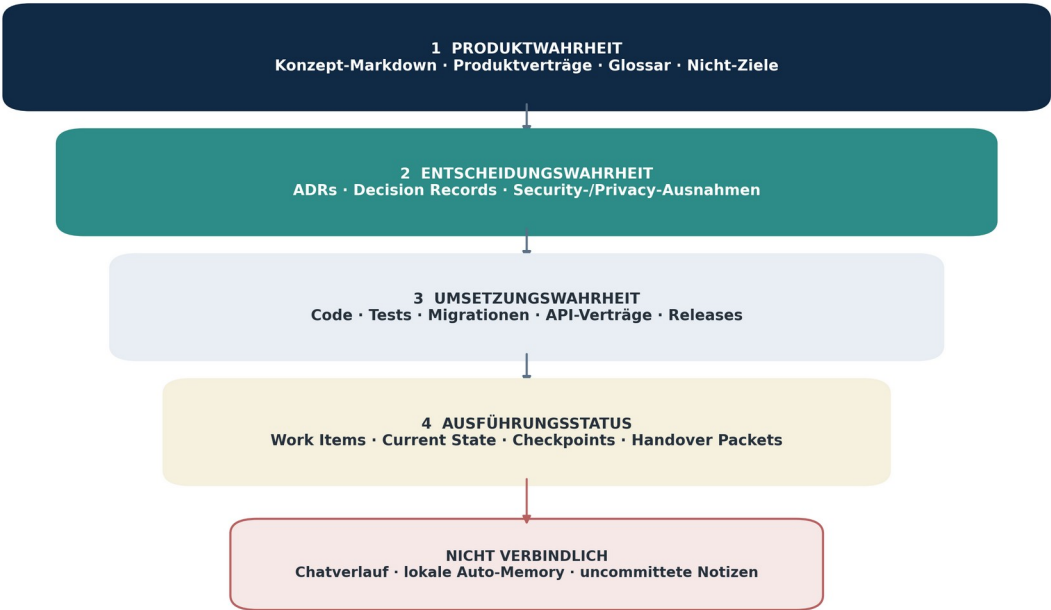
Claude Code, GitHub

Checkpoints & Bauplan

Repository-first Entwicklung, Context-sichere Sessions, kontinuierliche Checkpoints, GitHub Quality Gates, Agentenorchestrierung und ein phasenweiser Umsetzungsplan.

Gesichert. Fortsetzbar. Prüfbar. Schrittweise gebaut.

Repository-first Truth Model - was über Sessions hinweg verbindlich bleibt



Der Chat darf Arbeit ermöglichen, aber nie die einzige Quelle für Status, Entscheidung oder nächsten Einstieg sein.

ARBEITSBEZEICHNUNG ISMS Managed Service Platform
VERSION 1.0
STATUS Erstellt
STAND 22.07.2026
ABHÄNGIGKEITEN Dokument 00 bis 20B

Dokumentauftrag & Verbindlichkeit

Dokument 20C ist die verbindliche Entwicklungs-, Repository-, Checkpoint-, Handover-, GitHub- und Bauplanquelle. Es definiert, wie Claude Code und die Rollen aus Dokument 20B die vollständige Plattform über viele Context Windows, Sessions, Branches und Releases hinweg fortsetzbar, prüfbar und sicher umsetzen.

Steuerungsfeld	Festlegung
Dokument-ID	20C
Status	Erstellt - Version 1.0
Owner	Human Product Owner / CEO-Orchestrator / CTO / GitHub Steward / Project Memory / Quality Lead
Gültigkeit	Bis zur freigegebenen Nachfolgeversion
Umsetzungsstatus	Das Dokument definiert das Zielbetriebssystem. Konkrete Framework-, Hosting- und Toolversionen werden in Phase 0 durch Capability Check und ADR bestätigt.
Änderungskontrolle	Änderungen an Truth-Hierarchie, Checkpoint-Protokoll, Branchschutz, Human Gates, Quality Gates, Repository-Struktur oder Bauphasen benötigen Product-, Architecture-, Security-, Quality- und Continuity-Impactanalyse.

Inhalt

1 Auftrag · 2 Summary · 3 Entwicklungsverfassung · 4-10 Repository-Truth, Struktur, Claude-Konfiguration und Context Packs · 11-20 Work Packages, Sessions, Checkpoints und Wiederaufnahme · 21-35 GitHub, Tests, Agenten, Skills, Hooks und Rechte · 36-41 Umgebungen, Recovery, Gates und Metriken · 42-47 Bauphasen und Startprompts · 48-56 Demo, Akzeptanz, Entscheidungen, Annahmen, offene Fragen, Quellen und Änderungen.

1. Auftrag und Geltungsbereich

Dokument 20C übersetzt die Produktvision, die technische Architektur und die virtuelle KI-Firma in ein **konkret ausführbares Entwicklungsbetriebssystem**. Es legt fest, wie Claude Code und spezialisierte Agenten das Projekt über viele Sessions, Chats, Context Windows, Branches und Releases hinweg fortsetzbar, prüfbar und sicher umsetzen.

Das Dokument beantwortet insbesondere:

- Welche Dateien sind die verbindliche Projektwahrheit?
- Wie wird verhindert, dass Wissen nur in einem Chat oder Context Window existiert?
- Wie klein werden Aufgaben geschnitten, damit Claude sie zuverlässig abschließen kann?
- Wann muss innerhalb einer laufenden Session zwischengespeichert werden?
- Wie kann eine neue Session exakt dort fortsetzen, wo die vorherige aufgehört hat?
- Wie werden spezialisierte Agenten, Subagents, Worktrees und Reviews eingesetzt?
- Wie sehen Repository, Branches, Pull Requests, Tests, Dokumentation und Releases aus?
- Welche Entscheidungen darf Claude selbst treffen und wann muss der Mensch eingreifen?
- In welcher Reihenfolge wird die vollständige Plattform aufgebaut?

Dieses Dokument beschreibt **nicht den Produktumfang selbst**. Dafür gelten insbesondere Dokument 05 bis 19. Es beschreibt das Verfahren, mit dem dieser Umfang kontrolliert gebaut wird.

2. Executive Summary

Die zentrale Entscheidung lautet: **Das Repository ist das Projektgedächtnis. Der Chat ist nur eine Arbeitsoberfläche.**

Claude Code darf niemals darauf angewiesen sein, dass eine frühere Konversation vollständig verfügbar bleibt. Jede Session beginnt mit einem begrenzten, relevanten Kontextpaket und hinterlässt einen fortsetzbaren, versionierten Stand. Die Arbeit wird in kleine Work Packages zerlegt. Innerhalb eines Work Packages entstehen nicht nur am Ende, sondern regelmäßig **Micro Checkpoints**, **Verified Checkpoints** und bei Übergängen **Handover Checkpoints**.

Die Umsetzungsorganisation folgt dem Modell aus Dokument 20B:

- Der Human Product Owner verantwortet Vision und materiale Freigaben.
- Der CEO-/Orchestrator-Agent plant, zerlegt und stafft.
- Fach-, UX-, Architektur- und Engineering-Agenten erzeugen Artefakte.
- QA-, Security-, Privacy-, Product- und Code-Review-Agenten prüfen unabhängig.
- Project Memory und GitHub Steward halten Status und Repository konsistent.
- Claude Code arbeitet nicht als ein einzelner allwissender Agent, sondern als kontrollierte Organisation mit klaren Verträgen.

Der Build-Ansatz kombiniert:

1. **Markdown-Konzeptbibliothek** als fachliche Produktwahrheit,
2. **Architecture Decision Records und Decision Records** als Entscheidungswahrheit,
3. **Code, Tests und Migrationen** als Umsetzungswahrheit,
4. **Work Items, Current State, Checkpoints und Handovers** als Ausführungsstatus,
5. **Git und GitHub** als dauerhafte Versions-, Review- und Integrationsschicht,
6. **Claude-Code-Konfiguration, Skills, Rules, Hooks und Agentendefinitionen** als ausführbare Arbeitsregeln.

Die vollständige Plattform wird in kontrollierten Phasen aufgebaut. Die Produktvision wird nicht gekürzt; die Reihenfolge reduziert lediglich Risiko, Integrationsaufwand und Context-Überlastung.

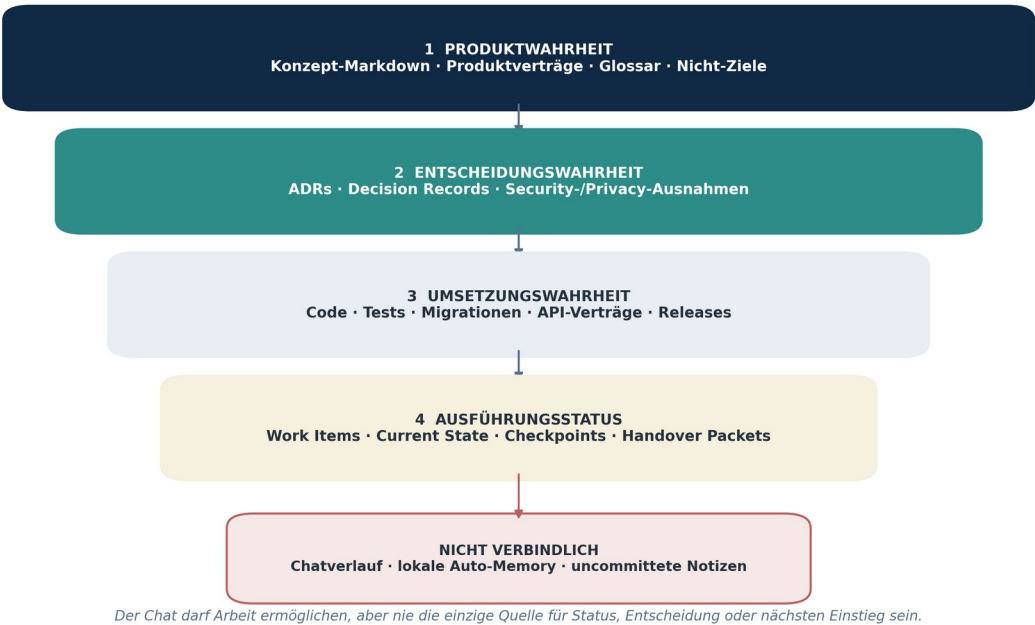
3. Entwicklungsverfassung

Für alle menschlichen und künstlichen Beteiligten gelten folgende Regeln:

1. **Repository vor Chat.** Eine relevante Entscheidung, ein Fortschritt oder ein Blocker gilt erst als gesichert, wenn er in einem versionierten Artefakt steht.
2. **Markdown vor PDF.** Die Markdown-Konzeptdateien sind die maschinenlesbare Quelle; PDFs und DOCX dienen Menschen, Präsentation und Review.
3. **Outcome vor Dateimenge.** Arbeit wird nach verifiziertem Nutzer- oder Systemergebnis bewertet, nicht nach erzeugten Dateien oder Zeilen Code.
4. **Kleine Work Packages.** Kein Agent erhält den Auftrag „Baue das gesamte Produkt“ oder „Implementiere das ganze Modul“.
5. **Kontinuierliche Checkpoints.** Zwischenspeicherung erfolgt während der Arbeit und nicht erst, wenn eine Session endet.
6. **Git ist Langzeitgedächtnis.** Native Session- oder Edit-Checkpoints sind zusätzliche Rückfallebenen, aber kein Ersatz für Commits und Pull Requests.
7. **Tests und Dokumentation entstehen mit der Funktion.** Sie sind kein späteres Aufräumprojekt.
8. **Builder und Reviewer sind getrennt.** Materiale Änderungen werden nicht ausschließlich vom erzeugenden Agenten freigegeben.
9. **Parallelität benötigt Isolation.** Schreibende Agenten verwenden getrennte Worktrees, Branches und klare Datei- oder Modulgrenzen.
10. **Kontext wird kuratiert.** Eine Session liest nur die für ihr Work Package notwendigen Konzept-, Architektur- und Statusdateien.
11. **Reversible Entscheidungen dürfen weiterlaufen.** Claude soll bei kleinen technischen Details sinnvoll entscheiden, dokumentieren und fortfahren.
12. **Materiale Entscheidungen stoppen kontrolliert.** Irreversible, kostenpflichtige, produktive, sicherheits-, datenschutz- oder rechtsrelevante Entscheidungen benötigen menschliche Freigabe.
13. **Keine geheimen Daten im Repository.** Secrets, Tokens, produktive Kundendaten und vertrauliche spätere Unternehmensinformationen werden nicht committed.
14. **Synthetische Demo-Daten.** Der Prototyp enthält realistische, aber erfundene Unternehmen, Rollen, Reports, Risiken, Services und Preise.
15. **Fortsetzung muss testbar sein.** Ein neuer Claude-Code-Chat muss anhand des Repository-Status ohne Rekonstruktion des alten Chats weiterarbeiten können.

4. Repository-first Truth Model

Repository-first Truth Model - was über Sessions hinweg verbindlich bleibt



Der Chat darf Arbeit ermöglichen, aber nie die einzige Quelle für Status, Entscheidung oder nächsten Einstieg sein.

Abbildung 1: Das Repository-first Truth Model trennt Produkt-, Entscheidungs-, Umsetzungs- und Statuswahrheit von nicht verbindlichem Chat- oder Auto-Memory-Kontext.

Die Projektwahrheit besitzt vier verbindliche Ebenen:

Ebene	Verbindliche Artefakte	Beantwortete Frage
Produktwahrheit	Konzept-Markdown, Glossar, Product Contracts, Nutzerreisen, Nicht-Ziele	Was soll gebaut werden und für wen?
Entscheidungswahrheit	ADRs, Decision Records, Security-/Privacy-Ausnahmen	Warum wurde eine konkrete Lösung gewählt?
Umsetzungswahrheit	Code, Tests, Schemas, Migrationen, API-Verträge, Releases	Wie verhält sich die gebaute Version tatsächlich?
Ausführungsstatus	GitHub Issues, Work Packages, Current State, Checkpoints, Handovers	Was ist fertig, was läuft und was kommt als Nächstes?

Nicht verbindlich sind:

- ein Chatverlauf ohne übertragene Artefakte,
- lokale, nicht versionierte Notizen,
- Auto-Memory ohne Review,
- uncommittete Zwischenstände,
- mündliche oder informelle Produktentscheidungen,
- automatisch erzeugte Zusammenfassungen, die nicht gegen Quellen geprüft wurden.

Bei Widersprüchen gilt folgende Reihenfolge:

1. freigegebene Produktentscheidung oder Konzept-Nachfolgeversion,
2. freigegebener ADR oder Decision Record,
3. aktueller, getesteter und gemergter Code,
4. aktives Work Package,
5. Handover- oder Sessionzusammenfassung,
6. Chatkontext.

Ein Widerspruch zwischen Konzept und implementiertem Verhalten darf nicht still gelöst werden. Er erzeugt ein Finding oder Decision Item.

5. Dokumentklassen im Repository

Dokumentklasse	Aufgabe	Lebenszyklus	Owner
Concept Document	Produktumfang und Anforderungen	versioniert, selten geändert	Product Lead / Domain Owner
Product Contract	konkreter Nutzeroutcome eines Features	pro Feature oder Journey	Product Lead
Architecture Decision Record	technische Entscheidung und Folgen	unveränderbar, später superseded	CTO / Architecture Lead
Decision Record	fachliche, UX-, Security- oder Betriebsentscheidung	unveränderbar, später superseded	benannter Decision Owner
Work Package	begrenzter Umsetzungsauftrag	geplant bis done/cancelled	Accountable Owner
Current State	komprimierter Projektstatus	laufend aktualisiert	Project Memory
Checkpoint	sicherer Zwischenstand	append-only oder referenziert	ausführender Agent / GitHub Steward

Dokumentklasse	Aufgabe	Lebenszyklus	Owner
Handover Packet	exakter Wiedereinstieg für neue Session/Rolle	pro Übergang	abgebender Owner
Test Evidence	Testfälle, Ergebnisse, Screenshots und Reports	pro Build/PR/Release	QA / Engineering
Finding	Abweichung, Fehler, Security- oder Qualitätsproblem	offen bis verifiziert geschlossen	Reviewer / Finding Owner
Release Record	Umfang, Version, Checks, Risiken und Rollback	pro Release	Release Steward
Runbook	wiederholbarer Betriebs- oder Entwicklungsprozess	versioniert	zuständiger Lead

6. Ziel-Repository-Struktur

Die folgende Struktur ist die verbindliche Ausgangsarchitektur. Claude darf sie begründet verfeinern, aber nicht ohne ADR grundlegend ersetzen.

Pfad	Zweck
/CLAUDE.md	kurze, globale Arbeitsregeln und Startreihenfolge
./claude/settings.json	projektweit teilbare, sichere Claude-Code-Einstellungen
./claude/rules/	path- oder themenspezifische Regeln
./claude/agents/	versionierte Rollen-/Subagent-Definitionen
./claude/skills/	wiederverwendbare Workflows wie Checkpoint, Review oder Handover
./claude/hooks/	geprüfte Skripte für Guardrails und Quality Gates
./github/ISSUE_TEMPLATE/	Feature-, Bug-, Security-, ADR- und Debt-Templates
./github/PULL_REQUEST_TEMPLATE.md	verpflichtende PR-Evidence und Checklisten
./github/CODEOWNERS	Reviewverantwortung nach Verzeichnis und Risiko
./github/workflows/	CI, Security, Tests, Docs, Releases und Qualitätsreports
/apps/web/	Web-Frontend
/apps/api/	API oder modulare Backend-Anwendung
/workers/	Workflow-, Connector-, Reporting- und Background-Worker
/packages/domain/	fachliche Domänenmodelle und gemeinsame Verträge
/packages/ui/	Designsystem und wiederverwendbare UI-Komponenten
/packages/contracts/	API-, Event-, Schema- und Output-Verträge
/packages/test-support/	Testfixtures, synthetische Daten und Test Utilities
/infra/	lokale, Test-, Staging- und spätere Deployment-Definitionen
/scripts/	reproduzierbare Bootstrap-, Checkpoint-, Test- und Maintenance-Skripte
/docs/concept/	Markdown-Fassungen der Konzeptsdokumente 00 bis 20C
/docs/product/	Product Contracts, Journeys, UI- und Demo-Spezifikationen
/docs/architecture/adr/	Architecture Decision Records
/docs/decisions/	fachliche und operative Decision Records
/docs/project/CURRENT_STATE.md	kompakter aktueller Gesamtstatus
/docs/project/WORK_QUEUE.md	priorisierte Queue und Abhängigkeiten
/docs/project/ACTIVE_WORK_PACKAGE.md	Verweis auf genau ein primäres aktives Paket
/docs/project/checkpoints/	versionierte Checkpoint Records
/docs/project/handovers/	Session- und Rollenübergaben; LATEST.md zeigt aktuellsten Einstieg
/docs/project/risks/	Entwicklungs-, Architektur-, Security- und Delivery-Risiken
/docs/quality/	Teststrategie, Definition of Done, Testmatrix, Findings
/docs/security/	Threat Model, Security Architecture, Ausnahmen und Reviews
/docs/releases/	Release Notes, Evidenz, Rollback und bekannte Einschränkungen
/demo/	synthetische Unternehmen, Seed-Daten und Demo-Skripte

Große generierte Dateien, Build Outputs, lokale Datenbanken, .env-Dateien und Secrets werden über .gitignore ausgeschlossen.

7. Zentrale Wahrheitsdateien

7.1 `CLAUDE.md`

CLAUDE.md ist die **Betriebsanweisung**, nicht das vollständige Konzept. Sie bleibt kurz und präzise. Zielgröße: etwa 80 bis 160 Zeilen; harte Leitplanke: keine unstrukturierte Wiederholung der Konzeptbibliothek.

Sie enthält mindestens:

- Produktdefinition in fünf bis acht Sätzen,
- Verweis auf docs/concept/00_MASTER_INDEX...md,
- verbindliche Startreihenfolge,
- Build-, Test-, Lint- und Typprüfbefehle,
- Repository- und Architekturprinzipien,
- Regeln für Daten, Secrets, Mandantenfähigkeit und Demo-Modus,
- Regeln für Work Packages, Checkpoints und Handovers,
- Stop Conditions und Human Gates,
- Definition of Done in Kurzform,
- Regel: relevante Entscheidungen und Fortschritt nie nur im Chat belassen.

7.2 `CURRENT\_STATE.md`

Diese Datei beantwortet auf maximal drei bis fünf Seiten:

- aktueller Produkt- und Implementierungsstand,
- zuletzt gemergte Version,
- aktive Phase und Meilenstein,
- aktives Work Package,
- gerade blockierte Themen,
- letzte wichtige Entscheidungen,
- Test- und Qualitätszustand,
- bekannte Risiken,
- exakter nächster Einstieg.

Sie ist kein historisches Log. Geschichte liegt in Git, Checkpoints, Releases und ADRs.

7.3 `LATEST.md`

docs/project/handovers/LATEST.md verweist auf das aktuellste Handover Packet und enthält zusätzlich eine maschinenlesbare Kurzfassung:

- Branch, Worktree und letzter Commit,
- Work Package ID,
- Status,
- nächste drei Aktionen,
- zwingend zu lesende Dateien,
- letzte Tests und ihr Ergebnis,
- Blocker und Human Gates.

7.4 `WORK\_QUEUE.md`

Die Work Queue ist eine komprimierte Repository-Sicht auf GitHub Issues und enthält:

- priorisierte Work Packages,
- Abhängigkeiten,
- Phase und Meilenstein,
- Owner und Reviewer,
- Risiko und geschätzte Größe,
- Status,
- Link oder ID zum GitHub Issue und PR.

GitHub bleibt das kollaborative Board. Die Markdown-Datei stellt lokale Fortsetzbarkeit sicher.

8. Claude-Code-Konfigurationsmodell

Das Projekt verwendet Claude Code in mehreren Konfigurationsebenen:

Mechanismus	Einsatz im Projekt	Nicht verwenden für
CLAUDE.md	globale, dauerhafte und kurze Regeln	lange Fachkonzepte oder historische Logs
.claude/rules/*.md	pfad- oder domänenspezifische Regeln	generische Wiederholungen
.claude/agents/*.md	spezialisierte Rollen mit Tool- und Scope-Grenzen	Projektstatus
.claude/skills/	wiederholbare Prozesse und Qualitätsroutinen	einmalige Produktentscheidung
.claude/settings.json	projektweit teilbare Einstellungen, Rechte und Hooks	Secrets oder persönliche Präferenzen
CLAUDE.local.md	persönliche lokale Hinweise, nicht committen	Teamregeln
Auto-Memory	optionale lokale Lernnotizen	Single Source of Truth oder Teamwissen
Session Transcript	kurzfristige Wiederaufnahme	dauerhafte Projektkontinuität

ENTSCHEIDUNG: Auto-Memory darf aktiviert sein, wird aber als nicht autoritative lokale Hilfe behandelt. Ein wichtiges Learning wird erst verbindlich, wenn es als Regel, ADR, Runbook oder dokumentierte Konvention ins Repository übernommen und reviewed wurde.

9. Path-scoped Rules und kontextarme Instruktionen

Große Projekte scheitern häufig nicht an fehlendem Wissen, sondern an zu viel gleichzeitig geladenem Wissen. Deshalb werden Regeln nach Pfaden aufgeteilt:

Regeldatei	Geltungsbereich	Beispielinhalt
architecture.md	apps/api/**, workers/**, packages/domain/**	Modulgrenzen, Events, Transaktionen, Fehlerverträge
frontend.md	apps/web/**, packages/ui/**	Designsystem, Accessibility, Loading-/Error-States
security.md	alle produktiven Dateien	Tenant Isolation, Secrets, Logging, Input Validation
testing.md	**/*.test.*, tests/**	Testpyramide, Fixtures, keine instabilen Sleeps
reporting.md	Reporting Engine	PPTX/PDF-Verträge, visuelle Regression, Quellenpflicht
integrations.md	Connectoren und Workflows	Idempotenz, Retries, Rate Limits, Dead Letter
docs.md	docs/**	Entscheidungstrennung, Quellen, Änderungslog
demo-data.md	demo/**	ausschließlich synthetische Daten, keine realen Markeninterna

Regeln werden so formuliert, dass ein Agent sie prüfen kann. „Schreibe guten Code“ ist unzureichend. „Jeder tenantgebundene Query benötigt einen expliziten Tenant Scope und einen Isolationstest“ ist prüfbar.

10. Kontextpakete pro Work Package

Kein Agent lädt zu Beginn alle PDFs, DOCX-Dateien und vollständigen Markdown-Dokumente. Für jedes Work Package wird ein kuratiertes CONTEXT_PACK.md erzeugt.

Ein Kontextpaket enthält:

- Ziel und Nicht-Ziel des Work Packages,
- relevante Product-Contract-Auszüge,
- betroffene Nutzerrollen und Journeys,
- relevante Daten- und API-Verträge,
- geltende ADRs und Security-Regeln,
- aktuelle Dateien und Tests,
- offene Findings,
- Akzeptanzkriterien,
- notwendige Links auf die vollständigen Quellen.

Leitplanken:

- Zielgröße: 8 bis 24 KB Text,
- nur relevante Konzeptabschnitte oder präzise Verweise,
- keine duplizierten Volltexte,
- automatische Erzeugung durch ein Repository-Skript,
- Hash oder Versionsangabe der genutzten Quelldokumente,
- Löschung oder Aktualisierung bei Abschluss des Work Packages.

Wenn eine Frage außerhalb des Kontextpakets liegt, liest der Agent gezielt die referenzierte Quelle. Er erfindet keine fehlende Produktentscheidung.

11. Work-Item- und Work-Package-Modell

Ein **Work Item** ist jede nachverfolgbare Arbeit. Ein **Work Package** ist die kleinste kontrolliert umsetzbare Einheit mit einem prüfbaren Outcome.

Feld	Pflichtinhalt
ID	stabile Kennung, z. B. WP-042
Titel	Outcome-orientiert, nicht rein technisch
Problem	welcher Nutzer- oder Systemnachteil behoben wird
Ziel	beobachtbares Ergebnis
Nicht-Ziele	bewusst ausgeschlossene Arbeit
Scope	Module, Dateien, Schnittstellen und Daten
Abhängigkeiten	vorgelagerte Work Packages, ADRs, Migrationen
Owner	genau ein Accountable Owner
Builder	ausführende Rolle oder Agent
Reviewer	unabhängige Rollen
Human Gates	erforderliche menschliche Freigaben
Acceptance Criteria	prüfbare fachliche und technische Kriterien
Test Plan	Unit, Integration, E2E, Security, Visual, Accessibility
Context Pack	zu lesende Quellen
Risk Class	niedrig, mittel, hoch, kritisch
Budget	Zeit-, Datei-, Turn-, Token- oder Iterationsgrenze

Feld	Pflichtinhalt
Checkpoint Plan	geplante Micro- und Verified Checkpoints
Deliverables	Code, Tests, Doku, Migrationen, Screenshots, Report
Stop Conditions	konkrete Gründe für Pause oder Eskalation
Done Evidence	PR, Commit, Tests, Demo und aktualisierte Doku

11.1 Größenleitplanke

Ein Work Package soll im Regelfall:

- einen klaren Nutzer- oder Architekturoutcome liefern,
- höchstens ein primäres Modul verändern,
- etwa fünf bis fünfzehn materielle Dateien betreffen,
- maximal eine Datenmigration oder einen externen Vertrag enthalten,
- in einer bis drei fokussierten Claude-Code-Sessions abschließbar sein,
- mindestens einen überprüfbaren Zwischenstand erlauben.

Wird diese Leitplanke überschritten, zerlegt der Orchestrator das Paket vor der Implementierung.

12. GitHub Issues, Projects und Meilensteine

GitHub Issues bilden den kollaborativen Arbeitsbestand. Empfohlene Issue Types:

- Product Feature,
- Technical Enabler,
- Bug,
- Security Finding,
- Privacy Finding,
- Architecture Decision,
- Technical Debt,
- Documentation,
- Research Spike,
- Release Task.

Empfohlene Project-Felder:

Feld	Werte oder Zweck
Status	Backlog, Ready, In Progress, In Review, Blocked, Done
Phase	0 bis 9 gemäß Bauplan
Priority	P0 bis P4
Risk	Low, Medium, High, Critical
Module	Product Domain oder technischer Bereich
Owner	verantwortliche Rolle
Reviewer	unabhängige Reviewrolle
Effort	XS, S, M, L; XL muss zerlegt werden
Target	Milestone oder Demo Release
Human Gate	None, Product, Security, Privacy, Cost, Production
Dependencies	Parent-/Sub-Issues oder verknüpfte Work Items

Issues sind mit Product Contract, ADR, Branch, Pull Request und Checkpoint verknüpft. Eine Taskliste im Chat ist nicht ausreichend.

13. Session-Boot-Sequenz

Session- und Chat-übergreifende Fortsetzung

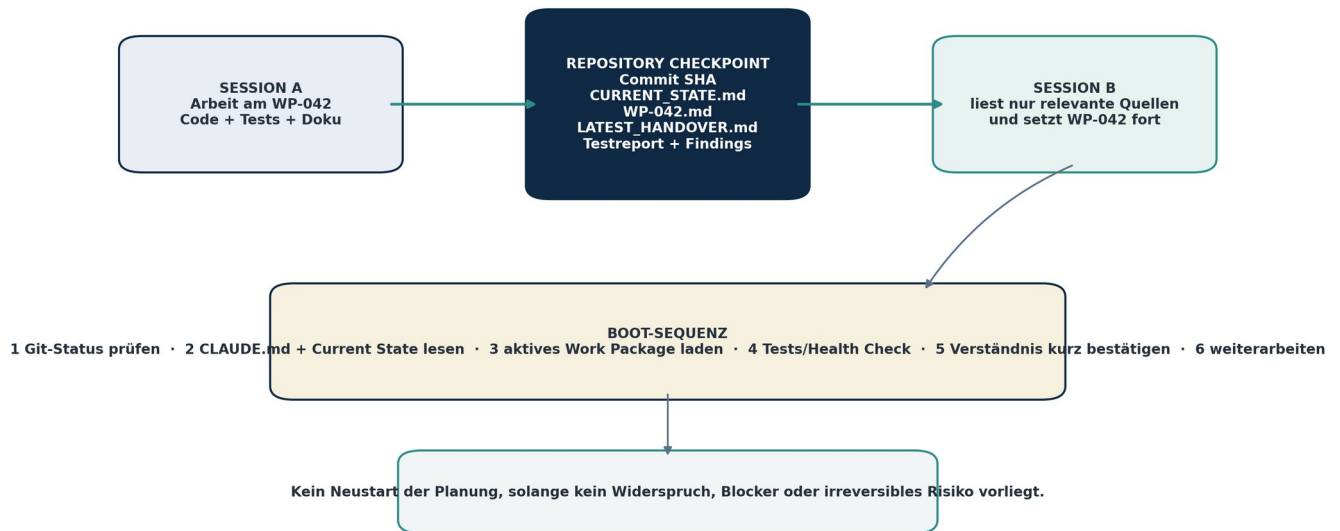


Abbildung 3: Ein Repository-Checkpoint ermöglicht die kontrollierte Fortsetzung eines Work Packages in einer neuen Claude-Code-Session.

Jede neue Claude-Code-Session führt vor materieller Arbeit folgende Sequenz aus:

1. Repository-Root und aktuellen Branch bestätigen.
 2. git status, letzte Commits und offenen Merge-/Rebase-Zustand prüfen.
 3. CLAUDE.md und relevante path-scoped Rules laden.
 4. docs/project/CURRENT_STATE.md lesen.
 5. docs/project/handovers/LATEST.md lesen.
 6. Aktives Work Package und sein Context Pack laden.
 7. Relevante Product Contracts, ADRs und Findings lesen.
 8. Abhängigkeiten und Human Gates prüfen.
 9. Setup-/Health-Check sowie vorhandene Tests ausführen, soweit wirtschaftlich.
 10. In maximal zehn Punkten Verständnis, aktuellen Stand und nächsten Schritt zusammenfassen.
 11. Ohne erneute Grundsatzplanung fortfahren, sofern kein Widerspruch oder Blocker besteht.
- Der Agent fragt nicht erneut nach Entscheidungen, die im Repository eindeutig dokumentiert sind.

14. Context-Budget-Strategie

Ein Context Window ist ein begrenzter Arbeitsraum, kein Projektarchiv. Das Projekt verwendet deshalb vier Schutzmechanismen:

1. **Context Packs** statt vollständiger Konzeptbibliothek,
2. **Subagents** für große Recherche-, Log- oder Reviewaufgaben mit komprimierter Rückgabe,
3. **regelmäßige Checkpoints** unabhängig vom sichtbaren Context-Stand,
4. **Sessionwechsel** nach einem verifizierten Zwischenstand statt maximaler Ausschöpfung.

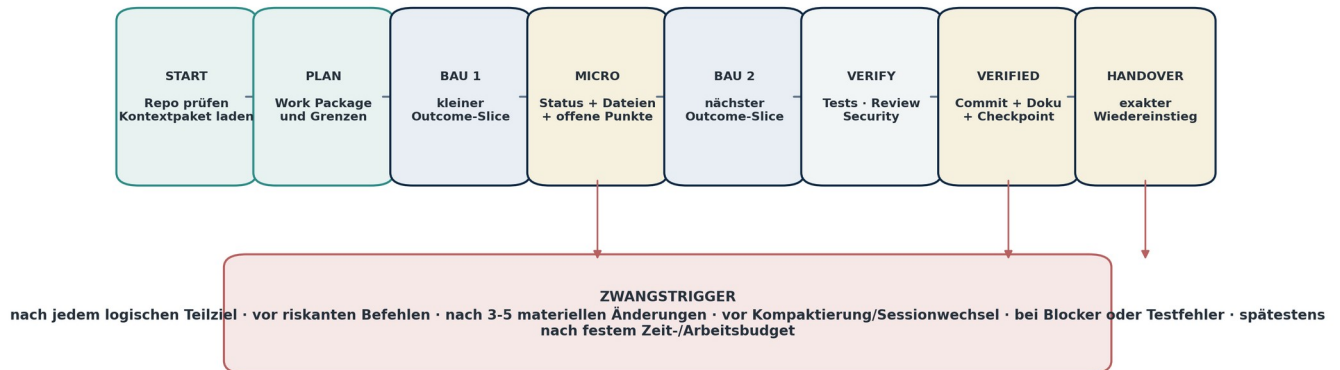
Da ein Agent den verbleibenden Context nicht immer exakt und rechtzeitig beurteilen kann, stützt sich die Sicherheit nicht allein auf Selbsteinschätzung. Stattdessen gelten harte Arbeitsgrenzen:

- nach jedem logischen Teilziel ein Micro Checkpoint,
- nach spätestens drei bis fünf materiellen Dateiänderungen Statusprüfung,
- nach einem größeren Testlauf oder Review ein Verified Checkpoint,
- vor jeder Kompaktierung oder bewussten Sessionübergabe ein Handover,
- bei unübersichtlicher Fehlersuche nach zwei bis drei Hypothesen einen Erkenntnis-Checkpoint,
- keine mehrstündige oder modulweite „One-Shot“-Implementierung ohne Zwischenergebnisse.

ENTSCHEIDUNG: Claude soll lieber eine Session früher kontrolliert beenden als einen großen ungesicherten Zustand im Context zu halten.

15. Checkpoint-Typen

Kontinuierliches Checkpoint-Protokoll - nicht erst am Session-Ende



Native Claude-Checkpoints ergänzen diese Routine, ersetzen aber weder Git noch den dokumentierten Projektstatus.

Abbildung 2: Checkpoints entstehen während der Arbeit nach festen Triggern und nicht erst am Ende einer Session.

15.1 Micro Checkpoint

Ein Micro Checkpoint entsteht nach einem logischen Teilziel. Er muss nicht zwingend ein Git-Commit sein, aktualisiert aber mindestens:

- Work-Package-Status,
- geänderte Dateien,
- erledigtes Teilziel,
- offene Fehler oder Annahmen,
- nächsten konkreten Schritt,
- Zeitpunkt und ausführende Rolle.

Micro Checkpoints werden in einer temporären, versionierbaren State-Datei oder als Abschnitt im aktiven Work Package geführt. Sie dürfen zusammengefasst werden, sobald ein Verified Checkpoint entsteht.

15.2 Verified Checkpoint

Ein Verified Checkpoint ist ein dauerhafter, wiederherstellbarer Zwischenstand. Voraussetzungen:

- Code und Dokumentation gespeichert,
- relevante Format-, Lint-, Typ- und Testprüfungen ausgeführt,
- Ergebnisse dokumentiert,
- keine wesentlich defekte Basis ohne Kennzeichnung,
- atomarer Git-Commit oder klarer PR-Zwischenstand,
- CURRENT_STATE.md oder Work Package aktualisiert,
- offene Risiken und nächster Einstieg dokumentiert.

15.3 Handover Checkpoint

Ein Handover Checkpoint wird vor Session-, Rollen-, Agenten- oder Worktreewechsel erstellt. Er enthält zusätzlich ein vollständiges Handover Packet.

15.4 Release Checkpoint

Ein Release Checkpoint enthält:

- freigegebenen Commit oder Tag,
- Release Notes,
- Test- und Security-Evidence,
- Migrationen,
- bekannte Einschränkungen,
- Rollback-Verfahren,
- Freigaben,
- aktualisierte Demo- oder Betriebsdokumentation.

16. Zwangstrigger für Checkpoints

Ein Checkpoint ist zwingend:

- nach Abschluss eines Acceptance-Criterion-Slices,
- nach drei bis fünf materiellen Dateiänderungen,
- vor Datenmigration, Mass-Refactoring oder destruktiven Befehlen,
- vor dem Wechsel zu einem anderen Modul,
- vor Übergabe an einen Reviewer,
- nach erfolgreichem Test- oder Fix-Zyklus,
- sobald ein Blocker oder ungeklärter Widerspruch entdeckt wird,
- vor /compact, /rewind, Sessionende oder bewusstem Chatwechsel,

- vor Merge, Release oder Deployment,
- wenn ein Agent seine Turn-, Zeit-, Kosten- oder Context-Grenze erreicht,
- wenn ein paralleler Agent auf denselben Vertrag oder dieselben Dateien angewiesen ist.

Ein Stop- oder vergleichbarer Hook soll prüfen, ob ein aktuelles Handover und ein frischer Status existieren. Der Hook darf das Beenden blockieren oder eine klare Warnung erzeugen, wenn materiale Änderungen ungesichert sind. Die genaue Hook-Konfiguration wird gegen die installierte Claude-Code-Version validiert.

17. Checkpoint-Automatisierung

Das Repository enthält ein reproduzierbares Skript, beispielsweise scripts/checkpoint, mit folgenden Aufgaben:

1. Work Package und Branch erkennen.
2. Git-Status und geänderte Dateien erfassen.
3. Tests oder einen gewählten Test-Scope ausführen.
4. Testergebnisse und bekannte Fehler speichern.
5. Checkpoint Record aus Template erzeugen.
6. CURRENT_STATE.md, Work Package und LATEST.md aktualisieren.
7. bei Verified Checkpoint einen atomaren Commit vorbereiten oder erstellen,
8. Geheimnisse, große Binärdateien und nicht erlaubte Dateien vor Commit prüfen,
9. einen maschinenlesbaren JSON-Status für nächste Agenten schreiben,
10. exakten Resume-Befehl oder Startprompt ausgeben.

Der Checkpoint-Prozess darf nicht automatisch einen fehlerhaften Zustand als „grün“ deklarieren. Ein bewusst roter Zwischenstand ist zulässig, wenn er isoliert, dokumentiert und nicht auf main gemergt ist.

18. Handover Packet

Jede Übergabe verwendet dasselbe Schema:

Feld	Inhalt
Handover ID	stabile Kennung und Zeitstempel
Work Package	ID, Titel und Link
Outcome	was erreicht werden soll
Status	Not Started, Active, Verifying, Blocked, Ready for Review, Done
Repository State	Branch, Worktree, Commit SHA, PR
Completed	konkret abgeschlossene Teilziele
Changed Files	gruppiert nach Funktion
Tests	Befehle, Ergebnis, Laufzeit, offene Flakes
Decisions	neue oder angewendete ADRs/Decision Records
Findings	offen, behoben, zurückgestellt
Risks	technische, fachliche, Security-, Privacy- und Delivery-Risiken
Human Gates	benötigt, offen oder erfüllt
Required Reading	nur zwingende Dateien für nächste Session
Exact Next Step	erste konkrete Aktion und erwartetes Ergebnis
Do Not Repeat	bereits geprüfte Hypothesen oder verworfene Varianten
Environment	notwendige Services, Seeds, Feature Flags und Ports
Recovery	wie auf letzten sicheren Stand zurückgekehrt wird

Ein Handover ohne „Exact Next Step“ ist unvollständig.

19. Wiederaufnahme in einem neuen Chat

Die nächste Session erhält nicht das gesamte alte Gespräch. Sie erhält einen kurzen Startauftrag:

Fortsetzungsprompt:

„Arbeite im vorhandenen Repository weiter. Lies zuerst CLAUDE.md, docs/project/CURRENT_STATE.md, docs/project/handovers/LATEST.md und das dort referenzierte aktive Work Package. Prüfe Git-Status, Branch, letzten Commit und vorhandene Tests. Fasse dein Verständnis und den exakten nächsten Schritt in höchstens zehn Punkten zusammen. Setze danach das aktive Work Package fort. Plane nicht das gesamte Projekt neu und wiederhole keine abgeschlossene Arbeit. Stoppe nur bei einem dokumentierten Widerspruch, einer irreversiblen Entscheidung, einem Sicherheits-/Datenschutzrisiko, einem Kostenrisiko oder einem fehlenden Human Gate. Aktualisiere während der Arbeit regelmäßig Checkpoints, Tests, Dokumentation und Handover.“

Nach dem Startprompt muss die Session auch dann fortsetzbar sein, wenn der vorherige Chat abrupt beendet wurde. Voraussetzung ist, dass der letzte Zwangsccheckpoint nicht zu lange zurückliegt.

20. Aktuelle Claude-Code-Fähigkeiten und bewusste Grenzen

Folgende Aussagen basieren auf der offiziellen Claude-Code-Dokumentation mit Recherchestand 22.07.2026 und müssen vor Implementierung gegen die konkret installierte Version geprüft werden:

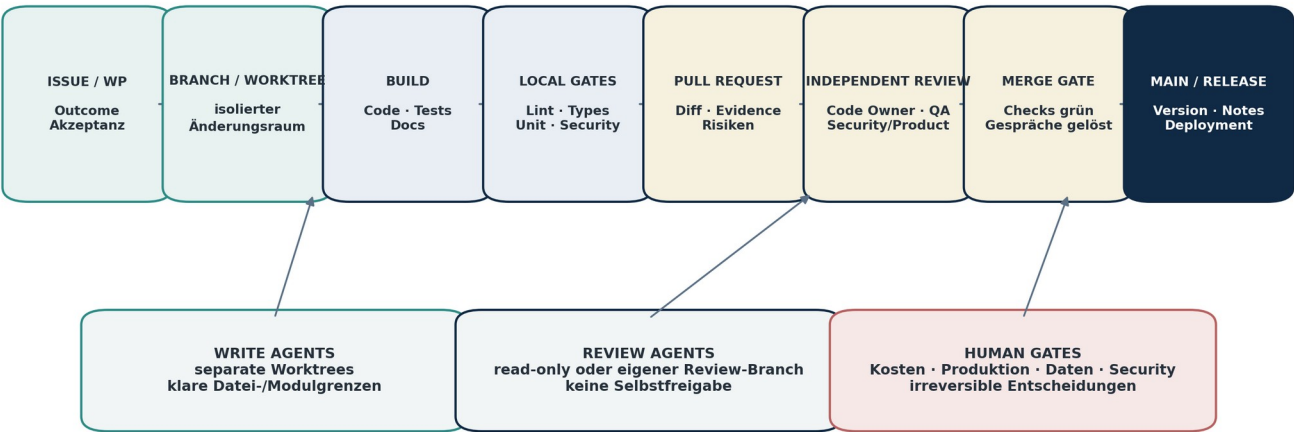
- Jede Session startet mit einem frischen Context Window; CLAUDE.md und Rules tragen dauerhafte Projektanweisungen über Sessions.
- Sessions können lokal gespeichert und fortgesetzt werden, sind aber kein teamweites oder maschinenübergreifendes Projektgedächtnis.
- Native Checkpoints verfolgen direkte Dateiänderungen durch Editierwerkzeuge, erfassen aber nicht zuverlässig jede Änderung durch Bash oder externe Prozesse und ersetzen daher Git nicht.

- Projektbezogene Subagents können in .claude/agents/ versioniert werden, besitzen eigene Context Windows und eingeschränkte Werkzeuge.
- Subagents eignen sich zur Context-Entlastung; verschachtelte Delegation ist begrenzt, weshalb zentrale Orchestrierung im Main Thread bleibt.
- Worktrees können parallele Sessions oder schreibende Agenten isolieren.
- Hooks können an Lifecycle- und Tool-Ereignisse gekoppelt werden und eignen sich für deterministische Guardrails.
- Agent Teams ermöglichen mehrere kommunizierende Sessions, sind zum Recherchestand jedoch experimentell und haben bekannte Einschränkungen.

ENTSCHEIDUNG: Die Kernfortsetzbarkeit darf nicht von experimentellen Agent Teams, lokalem Auto-Memory oder einer einzelnen Session abhängen. Sie basiert auf Git, Work Packages, Checkpoints und Handovers.

21. Branch- und Worktree-Strategie

GitHub Delivery Flow - isoliert bauen, unabhängig prüfen, kontrolliert integrieren



Main bleibt geschützt. Native Checkpoints sind lokales Undo; Git-Commits und Pull Requests sind die dauerhafte, kollaborative Historie.

Abbildung 4: Der GitHub Delivery Flow isoliert Änderungen, erzwingt unabhängige Reviews und integriert nur nach erfolgreichen Quality Gates.

21.1 Branches

Empfohlene Namenskonvention:

- feat/WP-042-morning-mission
- fix/BUG-117-tenant-scope
- sec/SEC-014-export-authz
- docs/DOC-032-risk-model
- chore/OPS-021-ci-cache
- spike/SPIKE-008-graph-query

main ist geschützt. Direkte Pushes sind nicht erlaubt. Materiale Änderungen laufen über Pull Requests.

21.2 Worktrees

Jeder parallel schreibende Agent oder jede parallele Claude-Session erhält einen eigenen Git Worktree. Regeln:

- ein Worktree pro Branch und Work Package,
- keine zwei Writer für dieselben Kern-Dateien,
- gemeinsame Verträge werden vor Parallelisierung versioniert,
- Änderungen werden nicht über Copy/Paste zwischen Worktrees übertragen, sondern über Commits oder Cherry-Picks,
- Worktrees ohne relevante Änderungen werden bereinigt,
- lokale .env- oder Tooldateien werden nur kontrolliert und niemals mit Secrets committed.

21.3 Parallelitätsmatrix

Arbeit	Parallel geeignet?	Bedingung
Frontend gegen stabilen API-Contract	ja	Mock/Contract vorhanden
Backend und Migration derselben Entität	eingeschränkt	ein Owner, sequenzierte Commits
Unit Tests zu stabilem Contract	ja	klare Akzeptanzkriterien
Security Review	ja, read-only	aktueller Diff und Threat Model
Zwei Agenten in derselben Datei	nein	außer bewusstes Pairing ohne Parallelwrites
Architekturentscheidung und Implementierung	teilweise	ADR vor materialer Umsetzung
Recherche konkurrierender Optionen	ja	getrennte Hypothesen, gemeinsame Synthese

Arbeit	Parallel geeignet?	Bedingung
Release und Migration	nein	zentraler Release Owner

22. Commit-Strategie

Commits sind klein, atomar und verständlich. Jeder Commit soll:

- einen kohärenten technischen oder fachlichen Schritt enthalten,
- kompiliert oder bewusst als nicht integrierbarer Zwischenstand markiert sein,
- keine Secrets, unkontrollierten Binärdateien oder irrelevanten Formatierungsänderungen enthalten,
- mit Work Item oder Finding verknüpft sein,
- passende Tests und Dokumentationsänderungen enthalten,
- eine verständliche Nachricht besitzen.

Empfohlenes Format:

- feat(decision-center): add role-aware morning mission cards [WP-042]
- fix(authz): enforce tenant scope on evidence export [SEC-014]
- test(graph): cover impact path cycle handling [WP-058]
- docs(adr): record workflow runtime choice [ADR-0012]

Ein Verified Checkpoint kann aus mehreren atomaren Commits bestehen. Ein Micro Checkpoint muss nicht committed werden.

23. Pull-Request-Vertrag

Jeder materiale Pull Request enthält:

- Work Package und Nutzeroutcome,
- Scope und Nicht-Ziele,
- geänderte Module und Verträge,
- Screenshots oder Demo-Schritte bei UI-Änderungen,
- Daten- und Migrationsauswirkungen,
- Security-, Privacy- und Tenant-Auswirkungen,
- Testbefehle und Ergebnisse,
- bekannte Einschränkungen,
- notwendige Feature Flags,
- Rollback- oder Recovery-Hinweise,
- Dokumentationsänderungen,
- offene Findings,
- Reviewer und Human Gates.

Ein PR wird als Draft geöffnet, sobald ein früher Review Mehrwert liefert. Er wird erst „Ready for Review“, wenn lokale Mindestgates erfüllt sind.

24. Branch Protection, Rulesets und CODEOWNERS

Für main gelten mindestens:

- Pull Request erforderlich,
- direkte Pushes und Force Pushes gesperrt,
- erforderliche Status Checks,
- mindestens eine unabhängige Freigabe,
- Code-Owner-Freigabe für kritische Bereiche,
- Auflösung aller Review-Kommentare,
- aktuelle Branch-Basis oder kontrollierte Merge Queue,
- lineare oder klar geregelte Historie,
- keine Umgehung durch Agentenrollen.

Beispielhafte CODEOWNERS-Bereiche:

Pfad	Reviewverantwortung
/docs/concept/	Product Lead + Domain Lead
/packages/domain/	Domain Lead + Architecture Lead
/apps/web/	Frontend Lead + UX/Accessibility Review
/apps/api/	Backend Lead + Architecture Review
/workers/integrations/	Integration Lead + Security Review
/infra/	Platform/DevOps + Security
/docs/security/	Security & Privacy Leads
/.github/, /.claude/	GitHub Steward + Security + Human Owner bei Berechtigungen

In einer privaten Ein-Personen-Prototypphase können tatsächliche GitHub-Benutzerrollen begrenzt sein. Die Agentenreviews werden trotzdem als Artefakte simuliert und später auf reale Reviewteams übertragbar gestaltet.

25. CI- und Quality-Gate-Pipeline

Die Pipeline ist risikobasiert und stufenweise:

25.1 Fast Gate

Bei jedem relevanten Push:

- Formatprüfung,
- Lint,
- Typprüfung,
- betroffene Unit Tests,
- Schema- und Contract-Validierung,
- Secret Scan,
- verbotene Datei- und Größenprüfung.

25.2 Pull-Request Gate

Zusätzlich:

- vollständige Unit Tests,
- Integrations- und Contract Tests,
- Datenbankmigrationstest,
- Tenant-Isolationstests,
- Dependency- und Supply-Chain-Prüfung,
- statische Securityanalyse,
- Accessibility-Prüfung für betroffene UI,
- visuelle Regression für zentrale Screens, PDF und PPTX,
- Dokumentations- und Linkprüfung,
- Demo-Seed-Validierung.

25.3 Release Gate

Zusätzlich:

- End-to-End-Flagship-Journeys,
- Smoke Tests in produktionsnaher Umgebung,
- Backup-/Restore- oder Migration-Rehearsal, soweit relevant,
- Performancebudgets,
- Security- und Privacy-Findings bewertet,
- SBOM oder Dependency Inventory,
- Release Notes und Rollback geprüft,
- Human Gate für Produktion oder externe Veröffentlichung.

Fehlgeschlagene Required Checks dürfen nicht durch einen Agenten umgangen werden.

26. Teststrategie

Tests werden aus Product Contracts, Risiken und Akzeptanzkriterien abgeleitet.

Testebene	Zweck	Beispiele
Unit	fachliche oder technische Logik isoliert	Risikoformeln, Statusübergänge, Priorisierung
Contract	Schnittstellen und Schemas	API, Events, Report Blocks, Connector Payloads
Integration	Zusammenspiel realer Komponenten	DB, Queue, Object Storage, Authz
Tenant Isolation	Verhinderung mandantenübergreifender Zugriffe	Queries, Exports, Jobs, Caches
E2E	vollständige Nutzerreise	Login bis Report und Decision Record
Security	Missbrauchs- und Angriffswege	Authz Bypass, Injection, File Upload, Secrets
Privacy	Zweckbindung und Datenrechte	Export, Löschung, Retention, Legal Hold
Accessibility	bedienbare UI	Tastatur, Semantik, Kontrast, Screenreader
Visual	Layoutstabilität	zentrale Screens, PDF, PPTX
Performance	Budgets und Skalierung	Graph Query, Portfolio, Reportgenerierung
Resilience	Ausfälle und Wiederaufnahme	Retry, Dead Letter, Worker Restart
Demo Determinism	reproduzierbare Präsentation	Seed, Zeit, Threat Event, Report Output

Flaky Tests werden als Findings erfasst und nicht dauerhaft ignoriert. Testdaten sind synthetisch und reproduzierbar.

27. Definition of Done

Ein Work Package ist erst fertig, wenn:

1. alle vereinbarten Acceptance Criteria erfüllt sind,
 2. Nutzer- oder Systemoutcome demonstrierbar ist,
 3. relevante Tests grün sind,
 4. Security-, Privacy- und Tenant-Auswirkungen geprüft sind,
 5. notwendige Migrationen reproduzierbar sind,
 6. Dokumentation und Product Contract aktualisiert sind,
 7. ADRs oder Decision Records erstellt wurden, wenn erforderlich,
 8. keine ungeklärten kritischen Findings bestehen,
 9. Review durch unabhängige Rolle dokumentiert ist,
 10. Demo- oder Abnahmeschritte vorhanden sind,
 11. CURRENT_STATE.md und Work Queue aktualisiert sind,
 12. ein Verified oder Release Checkpoint existiert,
 13. der nächste Workstream nicht auf Chatwissen angewiesen ist.
- „Code geschrieben“ ist kein Done-Zustand.

28. Architecture Decision Records und Decision Records

Ein ADR ist erforderlich bei:

- neuer Kerntechnologie,
- Änderung von Modulgrenzen,
- Datenbank- oder Graphstrategie,
- Authentifizierungs- oder Mandantenmodell,
- Workflow- oder Eventarchitektur,
- Hosting-, Deployment- oder Datenresidenzentscheidung,
- bedeutender Dependency,
- bewusstem Abweichen von Dokument 18 oder 19.

Ein Decision Record ist erforderlich bei:

- Materialer Produktpriorität,
- Zielgruppen- oder Journeyänderung,
- fachlicher Risiko- oder Reifegradlogik,
- UX-Kompromiss mit Nutzerwirkung,
- Security-/Privacy-Ausnahme,
- Preis- oder Servicepaketänderung,
- bewusster Verschiebung eines kritischen Features.

Records enthalten Kontext, Optionen, Entscheidung, Gründe, Folgen, Risiken, Owner, Datum und Supersede-Regel.

29. Dokumentationsbetrieb

Dokumentation wird in vier Rhythmen gepflegt:

- **mit jedem Work Package:** Product Contract, Work Item, Tests, relevante technische Doku,
- **bei jeder Entscheidung:** ADR oder Decision Record,
- **bei jedem Checkpoint:** Current State und Handover,
- **bei jedem Release:** Release Notes, bekannte Einschränkungen, Runbook und Demo-Stand.

Project Memory prüft regelmäßig:

- veraltete Links,
- widersprüchliche Statusangaben,
- erledigte Work Packages ohne aktualisierte Doku,
- Code ohne referenzierten Product Contract,
- ADRs ohne Implementierungsstatus,
- offene Findings ohne Owner,
- Konzeptänderungen ohne Aktualisierung von Dokument 00.

30. Agenten- und Subagentenabbildung in Claude Code

Die Rollen aus Dokument 20B werden als Kombination aus Main Session, Custom Subagents, Skills und separaten Worktree-Sessions umgesetzt.

Organisationsrolle	Primäre technische Abbildung
Human Product Owner	Mensch außerhalb der Agentenautomatisierung
CEO-/Orchestrator	Main Claude-Code-Session oder expliziter Coordinator Agent
Program Manager	Skill + Project-Management-Subagent
Product Lead	read/write Subagent für Product Contracts und Backlog
ISMS Domain Lead	fokussierter fachlicher Subagent, meist read-only Review
UX/UI Lead	Design-/Accessibility-Subagent, begrenzte UI-Schreibrechte
CTO/Architecture	Architecture-Subagent, ADR-Verantwortung
Frontend/Backend/Data/Integration	Worktree-isolierte Builder-Agenten
QA/Test	unabhängiger Test- und Review-Agent
Security/Privacy	read-only oder gezielt schreibender Review-Agent
Code Reviewer	read-only Review-Agent ohne Merge-Recht
Project Memory	Skill/Agent für State, Handover und Konsistenz
GitHub Steward	Agent/Workflow für Branch, PR, Labels und Checks
HR/Capability	Planungsagent; darf nur Vorschläge erzeugen

Projektbezogene Agentendefinitionen werden in `.claude/agents/` versioniert. Jede Definition verweist auf den Role Contract aus Dokument 20B und begrenzt Tools, Scope, Turns, Modellwahl und Stop Conditions.

31. Orchestrierungsregeln

Der CEO-/Orchestrator arbeitet nach folgenden Regeln:

1. Erst Work Package und Verträge klären, dann Agenten starten.
2. Nur Rollen aktivieren, deren Ergebnis einen erkennbaren Nutzen liefert.
3. Main Context nicht mit großen Logs oder Rechercheergebnissen überfüllen; dafür Subagents verwenden.
4. Schreibende Agenten in Worktrees isolieren.
5. Subagents erhalten vollständigen taskbezogenen Kontext, da sie nicht den gesamten Main-Chat erben.
6. Keine verschachtelte Agentenhierarchie voraussetzen; Delegation wird vom Main Thread koordiniert.
7. Parallelität stoppen, wenn Verträge instabil oder Dateien überlappend sind.

8. Reviews nicht an denselben Builder delegieren.
9. Ergebnisse über Artefakte und Commits integrieren, nicht nur über Zusammenfassungen.
10. Nach Integration einen Project-Memory-Checkpoint erzeugen.

32. Agent Teams – optionale, nicht kritische Erweiterung

Agent Teams können für unabhängige Forschung, Architekturvarianten, Frontend-/Backend-/Test-Workstreams oder konkurrierende Debugging-Hypothesen nützlich sein. Zum Recherchestand sind sie jedoch experimentell und mit erhöhten Token- und Koordinationskosten verbunden.

Daher gilt:

- Kernprozesse funktionieren ohne Agent Teams.
- Agent Teams werden zuerst in isolierten, synthetischen Work Packages erprobt.
- Keine produktkritische Freigabe hängt allein von Team-Messaging oder interner Taskliste ab.
- Ergebnisse müssen im Repository, in Issues und Checkpoints materialisiert werden.
- Bei bekannten Resume- oder Shutdown-Problemen wird auf Worktrees plus Main-Orchestrator zurückgefallen.
- Ein Team Lead ist nicht automatisch finaler Reviewer.

33. Skills

Mindestens folgende projektbezogene Skills werden vorgesehen:

Skill	Ergebnis
bootstrap-session	sichere Session-Boot-Sequenz und Verständniszusammenfassung
create-work-package	vollständiges Work Package mit Context Pack und Gates
checkpoint	Micro-, Verified- oder Handover-Checkpoint
handover	standardisiertes Handover Packet und LATEST.md
create-adr	ADR aus Optionen und Folgen
product-contract	Nutzeroutcome, Journey, Nicht-Ziele und Acceptance Criteria
review-pr	strukturierter Code-, Product-, Security- und QA-Review
security-review	Threat-, Tenant- und Datenflussprüfung
test-changed	risikobasierte Auswahl und Ausführung betroffener Tests
visual-qa	Screens, PDF und PPTX rendern und prüfen
release-candidate	Release Gate, Evidenz und Rollback Pack
project-memory-sync	Current State, Queue, Decisions und Handover konsolidieren
context-pack	relevante Quellen für ein Work Package zusammenstellen
resume-work	Repository prüfen und aktives Paket fortsetzen

Skills dürfen keine stillen Produktentscheidungen treffen. Sie automatisieren Verfahren.

34. Hooks und deterministische Guardrails

Hooks werden für Regeln eingesetzt, die nicht nur als Textanweisung existieren sollen.

Beispielhafte Einsatzpunkte:

Ereignis	Hook-Zweck
Session Start	Branch, Arbeitsverzeichnis, Required Files und Environment prüfen
Instructions Loaded	geladene Rule- und CLAUDE-Dateien protokollieren
PreToolUse Bash	gefährliche oder nicht erlaubte Befehle blockieren
PreToolUse Write/Edit	geschützte Pfade und Scope prüfen
PostToolUse Write/Edit	Formatierung, Lint oder Checkpoint-Alter prüfen
Subagent Start	Role Contract und taskbezogenen Kontext injizieren
Subagent Stop	Output Contract und Handover-Vollständigkeit prüfen
Stop	frischen Checkpoint, Tests und LATEST.md verlangen
Pre-Commit	Secrets, Größen, Format, Lint und Tests prüfen
Pre-Merge/CI	unabhängige Quality Gates erzwingen

Hooks werden als versionierte Skripte implementiert, getestet und mit Timeouts versehen. Experimentelle agentenbasierte Hooks sind kein Pflichtbestandteil; deterministische Command Hooks werden bevorzugt.

35. Berechtigungsmodell für Entwicklung

Claude Code arbeitet mit Least Privilege:

- Standardmäßig Lesezugriff auf das Repository,
- Schreibzugriff nur auf Work-Package-Scope,
- kein Zugriff auf .env, Secrets, private Schlüssel oder reale Kundendaten,
- Bash-Befehle über Allow-/Ask-/Deny-Regeln,

- git push, Releases, Deployment, Cloudänderungen und kostenpflichtige Befehle benötigen Freigabe,
- produktive Datenbanken und Produktionsumgebungen sind im Prototyp nicht verbunden,
- Security- und Review-Agenten erhalten bevorzugt read-only Rechte,
- Worktree-Agenten dürfen nur in ihrem isolierten Bereich schreiben,
- bypassPermissions oder vergleichbare gefährliche Modi werden nicht als Standard verwendet.

Lokale persönliche Einstellungen bleiben außerhalb des Repositories. Geteilte Einstellungen dürfen niemals Secrets enthalten.

36. Entwicklungsumgebungen

Umgebung	Zweck	Daten	Freigabe
Local	schnelle Entwicklung und Unit Tests	synthetisch, lokal	keine externe Freigabe
CI	reproduzierbare Checks	synthetische Fixtures	automatisiert
Preview	PR-bezogene UI-/Journey-Prüfung	synthetisch, isoliert	PR-basiert
Demo	stabile Vorführung des vollständigen Zielprodukts	kuratierte synthetische Unternehmen	Human Product Owner
Staging	produktionsnahe technische Validierung	synthetisch oder anonymisierte Testdaten	Quality + Security
Production	spätere reale Nutzung	echte Mandantendaten	außerhalb erster privater Version; formale Gates

Die Demo-Umgebung muss offline oder in einem Deterministic Demo Mode vorführbar sein, damit KI-API-, Netzwerk- oder Drittanbieterausfälle die Präsentation nicht zerstören.

37. Demo- und Seed-Datenbetrieb

Das Repository enthält mehrere synthetische Unternehmen mit unterschiedlichen Branchen, Reifegraden, Risiken, Zielen und Servicekombinationen. Jeder Seed besitzt:

- stabile IDs,
- dokumentierte Storyline,
- reproduzierbaren Zeitpunkt oder Time-Travel-Mechanismus,
- relevante Nutzerkonten und Rollen,
- Risiken, Controls, Maßnahmen, Evidenzen und Audits,
- Serviceverträge und Preisannahmen,
- Threat- oder Change-Events,
- erwartete KPIs und Reportausgaben,
- Reset-Skript,
- Validierungstests.

Ein Demo-Seed darf keine realen PwC-Daten, internen Preise, vertraulichen Templates oder behaupteten internen Prozesse enthalten.

38. Fehler- und Wiederherstellungsstrategie

38.1 Abbruch mitten in einer Session

- letzte versionierte Micro-/Verified-Checkpoint-Datei lesen,
- Git-Status prüfen,
- uncommittete Änderungen klassifizieren,
- Tests für betroffene Bereiche ausführen,
- Handover nachträglich aus Repository-Fakten rekonstruieren,
- keine Annahme treffen, dass der letzte Chat vollständig oder korrekt war.

38.2 Fehlerhafter Agenten-Commit

- Branch isolieren,
- nicht auf main mergen,
- Tests und Diff analysieren,
- über Git revert, fix-forward oder Worktree-Neustart entscheiden,
- Finding und Root Cause dokumentieren.

38.3 Widersprüchliche parallele Änderungen

- Integration pausieren,
- gemeinsame Verträge und ADRs bestimmen,
- einen Integrationsowner festlegen,
- Änderungen sequenzieren oder neu schneiden,
- keine automatisierte Konfliktauflösung bei fachlicher Bedeutung.

38.4 Beschädigter Current State

- Status aus Git, Issues, PRs, Checkpoints und Tests rekonstruieren,
- Project Memory Finding erstellen,
- CURRENT_STATE.md reviewen,
- Ursache beheben, beispielsweise fehlender Stop Hook oder zu großes Work Package.

38.5 Context-Verlust ohne Handover

- letztes Verified Checkpoint verwenden,
- ungesicherte Arbeit als unsicher behandeln,
- keine fiktive Fortsetzung aus Vermutungen,

- zuerst Recovery Work Item erstellen.

39. Human Gates

Menschliche Freigabe ist zwingend bei:

- Veränderung der Produktvision oder wesentlicher Nicht-Ziele,
- Kostenpflichtigen Buchungen oder Cloudressourcen oberhalb definierter Schwelle,
- produktivem Deployment,
- Datenlöschung oder irreversibler Migration,
- Zugriff auf reale oder vertrauliche Unternehmensdaten,
- Security- oder Privacy-Ausnahme mit materiellem Risiko,
- Änderung von Eigentums-, Lizenz- oder Übergaberechten,
- Veröffentlichung nach außen,
- Änderung zentraler Preis- oder Geschäftsmodellannahmen,
- Umgehung eines Required Quality Gates,
- Aktivierung gefährlicher Berechtigungsmodi,
- Aufgabe eines großen Produktmoduls.

Der Human Product Owner soll nicht für triviale reversible Technikdetails blockiert werden.

40. Stop Conditions

Claude oder ein Agent stoppt und eskaliert, wenn:

- Konzeptdokumente sich materiell widersprechen,
- Acceptance Criteria nicht eindeutig sind und mehrere Produktvarianten erzeugen,
- reale Daten, Secrets oder produktive Systeme betroffen wären,
- eine Änderung irreversibel oder destruktiv ist,
- Kosten außerhalb der freigegebenen Schwelle entstehen,
- Tenant Isolation, Authentifizierung oder Datenschutz unsicher sind,
- notwendige Tests nicht ausführbar sind und Risiko nicht begrenzt werden kann,
- ein kritischer Security-Fund offen ist,
- derselbe Fehler nach begrenzter Zahl sinnvoller Versuche ungelöst bleibt,
- das Work Package deutlich größer wird als geplant,
- Context oder Sessionzustand unübersichtlich wird und kein frischer Checkpoint besteht,
- Agenten parallele Änderungen nicht sicher integrieren können,
- ein Human Gate fehlt.

Stoppen bedeutet: sichern, dokumentieren, Ursache und Optionen darstellen. Es bedeutet nicht, unkommentiert aufzuhören.

41. Entwicklungsmetriken und Control Tower

Die virtuelle KI-Firma misst nicht nur Geschwindigkeit, sondern Fortsetzbarkeit und Qualität:

Kennzahl	Zweck
Lead Time pro Work Package	Flussgeschwindigkeit
Checkpoint Freshness	Risiko ungesicherter Arbeit
Handover Success Rate	Fortsetzbarkeit in neuer Session
Rework Rate	Qualität von Planung und Implementierung
Escaped Defects	Wirksamkeit der Gates
Review Finding Density	Risikobild und Lernbedarf
Test Stability	Zuverlässigkeit der Pipeline
Documentation Freshness	Konsistenz der Projektwahrheit
Context Pack Size	Schutz vor Kontextüberladung
Work Package Size Drift	Qualität der Zerlegung
Parallel Merge Conflict Rate	Eignung der Parallelisierung
Human Gate Wait Time	unnötige oder berechtigte Blockierung
Cost per Accepted Outcome	wirtschaftliche Agentennutzung
Demo Reliability	Vorfürbarkeit und deterministisches Verhalten

Anti-KPIs:

- Anzahl erzeugter Agentennachrichten,
- Anzahl Codezeilen,
- Zahl gestarteter Agenten,
- Zahl der Commits ohne Outcome,
- scheinbar hohe Geschwindigkeit bei steigendem Rework.

42. Bauplan – Phasenübersicht

Die vollständige Produktvision bleibt bestehen. Die Phasen definieren Integrationsreihenfolge und Quality Gates.

Phase	Ziel	Kern-Outcome
0	Repository und Entwicklungsbetriebssystem	fortsetzbares, getestetes Claude-/GitHub-Setup
1	Product Shell und Demo Foundation	Login, Rollen, Mandanten, Navigation, Seeds, Designsystem

Phase	Ziel	Kern-Outcome
2	Digitaler Unternehmenszwilling	Graphobjekte, Beziehungen, 360°-Ansichten, Historie
3	ISMS Core	Scope, Assets, Risiken, Controls, Maßnahmen, Evidence, Audits
4	Intelligence und Decision Center	Reifegrad, Threat Mapping, KPIs, Morning Mission, Simulationen
5	Collaboration und Reporting	Work Items, Entscheidungen, Freigaben, PDF/PPTX, Executive Experience
6	Managed-Service-Betrieb	Servicekatalog, Delivery, Value Ledger, Kunden-Lifecycle
7	Berater- und Portfolio-Operations	Portfolio, Kapazität, Skills, Termine, Reisen, Profitabilität
8	Integrationen und Automatisierung	Connector Hub, Workflow Designer, Health und Recovery
9	KI, Hardening und überzeugende Produktdemo	guarded AI, Security, Performance, vollständige Demo-Story

43. Phase 0 – Repository Bootstrap

Phase 0 ist abgeschlossen, wenn:

- Git-Repository initialisiert und remote gesichert ist,
- main geschützt oder lokal äquivalent geregelt ist,
- Ordnerstruktur und CLAUDE.md existieren,
- Konzept-Markdown 00 bis 20C unter docs/concept/ liegt,
- CURRENT_STATE.md, Work Queue, Work-Package- und Handover-Templates existieren,
- .claude/agents/, .claude/rules/, .claude/skills/ und sichere Settings angelegt sind,
- Checkpoint- und Context-Pack-Skripte als erste Version funktionieren,
- Issue- und PR-Templates vorhanden sind,
- Basis-CI mit Format, Lint, Typprüfung, Test, Secret Scan und Docs Check läuft,
- synthetische Demo-Datenregeln festgelegt sind,
- eine frische Claude-Code-Session anhand eines Handover Packets erfolgreich fortsetzt.

Der wichtigste Abnahmetest von Phase 0 ist kein Feature, sondern ein **Context-Loss-Drill**: Session A beginnt ein kleines Work Package, erstellt einen Verified Checkpoint und beendet. Session B setzt ohne alten Chat korrekt fort, schließt das Paket ab und erzeugt einen geprüften PR.

44. Phase 1 – Product Shell und Demo Foundation

Kernlieferungen:

- Web-App-Shell und neutrales Enterprise-Designsystem,
- Login-Simulation oder reale lokale Authentifizierung,
- mehrere synthetische Unternehmen,
- Rollenwechsel zwischen Executive, CISO, ISMS Manager, Berater und Admin,
- Mandantenkontext und Tenant Switch Guardrails,
- Navigation und universelles Seitenmuster,
- lokale Datenbank und Seed Reset,
- globale Suche als erster Stub,
- Feature Flags und Demo-Zeitsteuerung,
- Baseline für Accessibility, Visual Regression und E2E.

45. Phasen 2 bis 5 – Fachlicher Wertkern

45.1 Phase 2: Digital Twin

- kanonische Objekt- und Beziehungstypen,
- Unternehmen, Prozesse, Assets, Rollen und Lieferanten,
- 360°-Ansicht und Historie,
- Graph- oder relationale Graphprojektion,
- Quellen, Confidence und Data Quality,
- Impact Path und synthetische Szenarien.

45.2 Phase 3: ISMS Core

- Zielprofil, Scope und Schutzbedarf,
- Risiko- und Control-Lifecycle,
- Maßnahmen und Evidence,
- Policy- und Dokumentenlenkung,
- Lieferanten, Findings und Ausnahmen,
- Audit- und Management-Review-Grundlagen.

45.3 Phase 4: Intelligence und Decision Center

- Reifegrad, Risiko, Threat- und Control Intelligence,
- Morning Mission,
- Customer und Portfolio Decision Center,
- KPI Contracts,

- Routen- und Investitionssimulation,
- Decision Cards und Value Ledger.

45.4 Phase 5: Collaboration und Reporting

- Work Items, Freigaben und Decision Records,
- Notification Center,
- Management Reports,
- PPTX- und PDF-Engine,
- Executive Experience,
- nachvollziehbare Claims, Quellen und Snapshots.

46. Phasen 6 bis 9 – Skalierung und Differenzierung

46.1 Phase 6: Managed Services

- Service Definition, Offer, Instance und Run,
- Shared Responsibility,
- Servicekatalog und Paketkonfiguration,
- SLA-/SLO-Logik,
- Value Ledger und Service Review,
- Kunden-Onboarding, Expansion, Reduction und Exit.

46.2 Phase 7: Consultant Operations

- Portfolio Mission Control,
- Ressourcen-, Skill- und Kapazitätsplanung,
- Vor-Ort-, Audit-, Reise- und Kostenplanung,
- Coverage, Vertretung und Handover,
- Cost-to-Serve und ethische Opportunity-Erkennung.

46.3 Phase 8: Integrationen und Automatisierung

- Connector Framework,
- Entra-, Ticketing-, Security-, Cloud- und Dokumenten-Mocks,
- Sync, Webhook, Reconciliation und Health,
- Workflow Designer,
- Human Gates, Retry, Dead Letter und Replay,
- Automatisierungswirkung.

46.4 Phase 9: KI und Hardening

- Model Gateway und AI Control Plane,
- guarded Summaries, Reports, Risikoerklärungen und Empfehlungen,
- RAG mit Quellen und Tenant Scope,
- Deterministic Demo Mode,
- Security-, Privacy-, Performance- und Resilience-Hardening,
- vollständige 5- bis 10-minütige Demo-Story,
- Übergabe- und Betreiberpaket.

47. Erster ausführbarer Claude-Code-Startprompt

„Du übernimmst die strukturierte Umsetzung der ISMS Managed Service Platform auf Grundlage der Konzeptbibliothek im Repository.

1. Lies zuerst CLAUDE.md, docs/concept/00_MASTER_INDEX_UND_PROJEKTVERFASSUNG_v1.0.md und docs/concept/20C_CLAUDE_CODE_GITHUB_CHECKPOINTS_BAUPLAN_v1.0.md.
2. Prüfe danach die vorhandene Repository-Struktur, Git-Historie, docs/project/CURRENT_STATE.md, Work Queue und das aktuelle Handover.
3. Fasse dein Verständnis von Produkt, Nutzern, vollständiger Zielvision, Entwicklungsverfassung, Agentenorganisation und Phase 0 zusammen.
4. Melde nur tatsächliche Widersprüche oder blockierende Voraussetzungen. Erfinde keine Produktentscheidungen.
5. Stelle, falls noch nicht vorhanden, einen begrenzten Bootstrap-Plan für Phase 0 mit kleinen Work Packages und klaren Acceptance Criteria.
6. Lege Repository, CLAUDE.md, .claude-Struktur, Projektstatus, Templates, sichere Settings, Testgrundlage und CI schrittweise an.
7. Arbeite in kleinen, prüfbareren Outcomes. Erzeuge während der Arbeit regelmäßig Micro und Verified Checkpoints; warte damit nicht bis zum Ende der Session.
8. Aktualisiere nach jedem größeren Teilziel Work Package, Tests, Dokumentation und CURRENT_STATE.md.
9. Nutze spezialisierte Agenten nach Dokument 20B, aber halte Orchestrierung und finale Integration zentral. Schreibende parallele Agenten arbeiten isoliert in Worktrees.
10. Frage nur bei blockierenden, irreversiblen, sicherheits-, datenschutz-, kosten- oder produktionsrelevanten Entscheidungen. Bei reversiblen technischen Details entscheide sinnvoll, dokumentiere und arbeite weiter.
11. Führe vor Merge vor Übergabe alle relevanten Tests und Quality Gates aus.
12. Beende jede Session mit einem Handover Packet, einem genauen nächsten Schritt und einem sicheren Repository-Zustand.

Beginne mit Analyse und Verständniszusammenfassung. Fahre anschließend mit dem ersten nicht blockierten Phase-0-Work-Package fort.“

48. Verbindliche Demo- und Abnahmeszenarien

1. Repository Bootstrap aus leerem Projektordner.
2. Claude liest nur Index, 20C und relevante Quellen statt alle PDFs.
3. Orchestrator erstellt kleine Work Packages.
4. Feature-Builder arbeitet in eigenem Worktree.
5. Security-Reviewer besitzt read-only Scope und erzeugt Findings.

6. Context-Loss-Drill zwischen zwei Sessions.
7. Micro Checkpoint nach einem logischen Teilziel.
8. Verified Checkpoint mit Tests und atomarem Commit.
9. Stop Hook erkennt fehlendes Handover.
10. Neuer Chat liest LATEST.md und setzt exakt fort.
11. PR zeigt Product-, Test-, Security- und Dokumentationsevidence.
12. Required Check verhindert Merge bei Tenant-Isolationstestfehler.
13. CODEOWNERS oder simulierte Reviewmatrix fordert unabhängige Freigabe.
14. Parallel arbeitende Frontend- und Backend-Agenten nutzen versionierten Contract.
15. Konflikt in gemeinsamem Vertrag stoppt Parallelität kontrolliert.
16. Projektgedächtnis rekonstruiert Status aus Git und Issues.
17. Auto-Memory enthält ein Learning, das erst nach Review als Rule übernommen wird.
18. Native /rewind- oder Sessioncheckpoint-Funktion wird demonstriert, aber Git bleibt Recovery-Basis.
19. Demo-Seed wird reproduzierbar zurückgesetzt.
20. Phase-0-Release enthält Release Record und funktionsfähigen Continuity Drill.

49. Globale Akzeptanzkriterien

1. Das Repository besitzt eine eindeutige Truth-Hierarchie.
2. Markdown und nicht PDF ist die maschinenlesbare Konzeptquelle.
3. CLAUDE.md bleibt kurz, konkret und verweist auf weitere Quellen.
4. Path-scoped Rules reduzieren irrelevanten Context.
5. Jedes materiale Work Item besitzt ein Work Package mit Owner, Scope und Acceptance Criteria.
6. Work Packages sind klein genug für kontrollierte Checkpoints.
7. Nach jedem logischen Teilziel kann ein Micro Checkpoint erzeugt werden.
8. Verified Checkpoints enthalten Tests, Commit, Status und nächsten Einstieg.
9. Handover Packets enthalten Branch, Commit, Dateien, Tests, Risiken und Exact Next Step.
10. Eine neue Session kann ohne alten Chat fortsetzen.
11. Chat, Transcript und Auto-Memory sind nicht die einzige Projektquelle.
12. Git wird als dauerhafte Historie verwendet.
13. main ist vor direkten oder ungeprüften Änderungen geschützt.
14. Parallele Writer arbeiten in getrennten Worktrees.
15. Builder und finaler Reviewer sind getrennt.
16. PRs enthalten fachliche, technische, Test-, Security- und Doku-Evidence.
17. Required Checks können nicht still umgangen werden.
18. CODEOWNERS oder eine äquivalente Reviewmatrix deckt kritische Pfade ab.
19. Secrets und reale Kundendaten werden nicht committed oder in Agentenkontext geladen.
20. Demo-Daten sind synthetisch und reproduzierbar.
21. Agenten besitzen begrenzte Tools und einen Role Contract.
22. Orchestrierung hängt nicht von experimentellen Agent Teams ab.
23. Hooks erzwingen kritische Regeln deterministisch, soweit technisch möglich.
24. Tests decken Hauptwege, Fehlerfälle, Tenant Isolation, Security, Accessibility und Demo ab.
25. Dokumentation ist Teil der Definition of Done.
26. ADRs und Decision Records sind versioniert und nachvollziehbar.
27. Human Gates schützen irreversible, kosten-, daten- und produktionsrelevante Entscheidungen.
28. Stop Conditions erzeugen einen gesicherten Status statt stillen Abbruch.
29. Phase 0 enthält einen erfolgreichen Context-Loss- und Resume-Test.
30. Die Bauphasen erhalten die vollständige Produktvision und ordnen nur die Umsetzung.
31. Ein Agent kann anhand der Repository-Artefakte erklären, was fertig ist und was als Nächstes kommt.
32. Die virtuelle KI-Firma erzeugt weniger Rework als ein unstrukturierter Einzelagentenprozess.

50. Festgelegte Entscheidungen

- **D20C-001:** Das Git-Repository ist das zentrale Projektgedächtnis; Chats sind nicht autoritativ.
- **D20C-002:** Markdown-Konzeptdateien sind die maschinenlesbare Quelle; PDF und DOCX sind Derivate.
- **D20C-003:** Produkt-, Entscheidungs-, Umsetzungs- und Statuswahrheit werden getrennt verwaltet.
- **D20C-004:** CLAUDE.md bleibt kurz und enthält keine vollständige Konzeptbibliothek.
- **D20C-005:** Kontext wird pro Work Package kuratiert.
- **D20C-006:** Arbeit wird in kleine outcome-orientierte Work Packages zerlegt.
- **D20C-007:** Checkpoints entstehen kontinuierlich und nicht nur am Session-Ende.
- **D20C-008:** Es gibt Micro, Verified, Handover und Release Checkpoints.
- **D20C-009:** Git-Commits und PRs sind dauerhafte Historie; native Claude-Checkpoints sind ergänzendes Undo.
- **D20C-010:** Jede Session beginnt mit einer festen Boot-Sequenz.
- **D20C-011:** Jede Session oder Rollenübergabe erzeugt ein Handover Packet.
- **D20C-012:** Eine neue Session plant das Projekt nicht neu, solange kein Widerspruch oder Blocker besteht.
- **D20C-013:** main wird geschützt und materiale Änderungen laufen über Pull Requests.
- **D20C-014:** Parallele Writer verwenden isolierte Worktrees und getrennte Änderungsgrenzen.
- **D20C-015:** Builder und finaler Reviewer sind bei materialer Arbeit getrennt.
- **D20C-016:** Required Quality Gates dürfen von Agenten nicht umgangen werden.
- **D20C-017:** Auto-Memory und Sessiontranscripts werden nicht als Teamwahrheit verwendet.
- **D20C-018:** Experimentelle Agent Teams sind optional und nicht Voraussetzung der Kernorganisation.
- **D20C-019:** Project Memory und GitHub Steward sind permanente Kernverantwortungen.
- **D20C-020:** Hooks werden für deterministische Guardrails eingesetzt.
- **D20C-021:** Secrets und reale Unternehmensdaten bleiben außerhalb des Repositories und der Demo.

- **D20C-022:** Die Demo besitzt einen deterministischen, reproduzierbaren Daten- und Resetmodus.
- **D20C-023:** Phase 0 wird durch einen erfolgreichen Context-Loss-Drill abgenommen.
- **D20C-024:** Die vollständige Produktvision bleibt erhalten; Phasen priorisieren nur die Baufolge.

51. Begründete Annahmen

- **A20C-001:** Claude Code oder ein vergleichbares Coding-Agent-System kann projektbezogene Instructions, Rules, Skills, Hooks und spezialisierte Agenten aus versionierten Dateien laden.
- **A20C-002:** Ein artifact-first Prozess reduziert Context-Verlust und widersprüchliche Entscheidungen erheblich.
- **A20C-003:** Kleine Work Packages erhöhen Abschlussquote und Reviewqualität.
- **A20C-004:** Ein fester Checkpoint-Rhythmus ist zuverlässiger als alleinige Selbsteinschätzung des verbleibenden Context Windows.
- **A20C-005:** GitHub Issues und Projects reichen für die erste Entwicklungsorganisation aus.
- **A20C-006:** In der privaten Prototypphase können unabhängige Agentenreviews reale Teamreviews teilweise simulieren.
- **A20C-007:** Spätere Unternehmensübernahme kann Repository-, Branding-, Preis- und Rollenmodelle erweitern, ohne die Kernarchitektur zu ersetzen.
- **A20C-008:** Der modulare Monolith aus Dokument 18 ist für die ersten Phasen schneller und sicherer als frühe Microservices.
- **A20C-009:** Worktrees sind für parallele schreibende Agenten ausreichend isolierend, wenn gemeinsame Verträge stabil sind.
- **A20C-010:** Die konkrete Claude-Code-Version kann Features, Konfigurationsfelder und Hookevents verändern; daher wird vor Bootstrap ein Capability Check durchgeführt.
- **A20C-011:** Die meisten Sessions können mit einem Context Pack von unter 24 KB starten.
- **A20C-012:** Ein lokaler oder cloudbasierter CI-Runner kann die erforderlichen Tests und Dokumentenprüfungen ausführen.
- **A20C-013:** Eine realistische Demo kann weitgehend mit synthetischen Daten und Mock-Integrationen überzeugen.
- **A20C-014:** Der Human Product Owner möchte bei blockierenden und irreversiblen Fragen entscheiden, nicht bei jedem technischen Detail.

52. Offene Fragen

- **O20C-001:** Welche konkrete Programmiersprache, Framework- und Package-Manager-Kombination wird beim Bootstrap final gewählt?
- **O20C-002:** Wird das Repository zunächst privat unter einem persönlichen GitHub-Konto oder einer Organisation angelegt?
- **O20C-003:** Welche GitHub-Funktionen und Schutzregeln stehen im gewählten Plan tatsächlich zur Verfügung?
- **O20C-004:** Welche Claude-Code-Version und welche Features sind beim Start installiert?
- **O20C-005:** Werden Agent Teams in Phase 0 getestet oder bewusst bis zu einer stabileren Version verschoben?
- **O20C-006:** Welche Hooks können in der gewählten Version zuverlässig blockierend arbeiten?
- **O20C-007:** Soll autoMemoryEnabled projektweit deaktiviert oder nur als nicht autoritativ dokumentiert werden?
- **O20C-008:** Welche maximale Zeit-, Token- oder Kostenbudgets gelten pro Work Package und Agent?
- **O20C-009:** Wer übernimmt in der Ein-Personen-Phase reale PR-Freigaben, wenn GitHub eine menschliche Approval verlangt?
- **O20C-010:** Welche CI-Ressourcen dürfen kostenpflichtig verwendet werden?
- **O20C-011:** Wird eine lokale Containerumgebung oder ein cloudbasierter Dev Container Standard?
- **O20C-012:** Welche Datenbank- und Graphumsetzung aus Dokument 18 wird final gewählt?
- **O20C-013:** Welche UI- und E2E-Testwerkzeuge werden eingesetzt?
- **O20C-014:** Wie werden PDF- und PPTX-Visual-Regressionen in CI wirtschaftlich ausgeführt?
- **O20C-015:** Welche Mindestabdeckung muss ein Work Package vor Merge erreichen, ohne Fehlanreize durch reine Prozentziele zu erzeugen?
- **O20C-016:** Welche späteren PwC- oder Enterprise-Richtlinien müssen bei einer Übergabe in Managed Settings, CODEOWNERS und CI ergänzt werden?
- **O20C-017:** Soll das Repository ein Monorepo bleiben, wenn Integrationen und Reporting stark wachsen?
- **O20C-018:** Ab wann ist eine echte Staging-Umgebung gegenüber lokaler Demo wirtschaftlich sinnvoll?

53. Ideen für später

- visuelles Development Control Tower mit Work Packages, Agenten, Context Budget, Gates und Kosten,
- automatisch generierte Context Packs anhand Code Ownership und Dependency Graph,
- semantische Konsistenzprüfung zwischen Konzept, ADRs, Code und Tests,
- automatische Handover-Qualitätsbewertung,
- agentenübergreifende Lernbibliothek mit reviewter Promotion in Skills,
- reproduzierbare Cloud Development Environments pro Work Package,
- Merge Queue mit risikobasierter Priorisierung,
- automatisch erzeugte Release-Demos und Storyboards,
- Simulation verschiedener Agententeams nach Kosten, Zeit und Rework,
- Policy-as-Code für Human Gates und Agentenrechte,
- kryptografisch signierte Decision- und Release-Records,
- digitale Übergabebox für spätere Übernahme durch PwC oder andere Betreiber,
- GitHub App zur Anzeige von ISMS-spezifischen Product- und Security-Gates,
- automatische Erkennung veralteter Konzeptabschnitte durch Codeänderungen,
- Work-Package-Generator aus Nutzerjourneys und Product Contracts,
- agentengestützte Chaos- und Recovery-Übungen für Entwicklungsorganisation und Plattform.

54. Abhängigkeiten

Dokument 20C baut insbesondere auf:

- Dokument 00: Master-Index, Projektverfassung und Single Source of Truth,
- Dokument 01: Vision, Nutzen und Business Case,
- Dokument 03 bis 06: Rollen, Journeys, Produktumfang und UX,
- Dokument 07 bis 10: Digital Twin, ISMS Core, Intelligence und Decision Center,
- Dokument 11 und 12: Zusammenarbeit, Work Items, Handover und Reporting,
- Dokument 13 bis 17: Managed Services, Beraterbetrieb, Onboarding, Integrationen und Workflows,
- Dokument 18: technische Architektur,
- Dokument 19: Security, Privacy, Rechte und Auditierbarkeit,
- Dokument 20A: KI-Funktionen und Guardrails,
- Dokument 20B: virtuelle KI-Firma und Role Contracts.

Nach Freigabe dieses Dokuments muss Dokument 00 aktualisiert werden. Anschließend kann Phase 0 beginnen.

55. Quellenregister und Capability Notes

Die folgenden externen Quellen wurden ausschließlich für aktuelle Tool- und GitHub-Fähigkeiten verwendet. Produktentscheidungen dieses Dokuments sind eigene Projektentscheidungen und keine Herstellerempfehlungen.

Quelle	Verwendeter Aspekt	Abrufstand
Anthropic Claude Code Docs – Memory	frische Context Windows, CLAUDE.md, Rules, Auto-Memory und deren Grenzen	22.07.2026
Anthropic Claude Code Docs – Sessions	Resume, lokale Sessions und Sessionverwaltung	22.07.2026
Anthropic Claude Code Docs – Checkpointing	native Edit-Checkpoints, Rewind und Abgrenzung zu Git	22.07.2026
Anthropic Claude Code Docs – Subagents	projektbezogene Subagents, eigene Context Windows, Toolgrenzen	22.07.2026
Anthropic Claude Code Docs – Agent Teams	experimenteller Status, Team Lead, Taskliste und Grenzen	22.07.2026
Anthropic Claude Code Docs – Worktrees	isolierte parallele Sessions und Subagent Worktrees	22.07.2026
Anthropic Claude Code Docs – Hooks	Lifecycle Hooks und deterministische Guardrails	22.07.2026
Anthropic Claude Code Docs – Permissions/Settings	Rechte, Sandbox, Konfigurationssscopes	22.07.2026
GitHub Docs – Protected Branches / Rulesets	PR-, Review-, Status-Check- und Branchschutz	22.07.2026
GitHub Docs – CODEOWNERS	automatische Reviewverantwortung für Pfade	22.07.2026
GitHub Docs – Issues and Projects	Work Items, Felder, Sub-Issues und Projektansichten	22.07.2026
GitHub Docs – Pull Request Reviews / Status Checks	unabhängige Reviews und automatisierte Qualitätsprüfungen	22.07.2026

Offizielle Referenzen:

- <https://code.claude.com/docs/en/memory>
- <https://code.claude.com/docs/en/sessions>
- <https://code.claude.com/docs/en/checkpointing>
- <https://code.claude.com/docs/en/sub-agents>
- <https://code.claude.com/docs/en/agent-teams>
- <https://code.claude.com/docs/en/worktrees>
- <https://code.claude.com/docs/en/hooks>
- <https://code.claude.com/docs/en/permissions>
- <https://code.claude.com/docs/en/settings>
- <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>
- <https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners>
- <https://docs.github.com/en/issues/planning-and-tracking-with-projects>
- <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests/about-pull-request-reviews>

Herstellerfunktionen können sich ändern. Vor der tatsächlichen Repository-Initialisierung führt Claude einen Capability Check gegen die installierte Version aus und dokumentiert Abweichungen als ADR oder Bootstrap Finding.

56. Änderungsprotokoll

Version	Datum	Änderung	Status
1.0	22.07.2026	Erstfassung des Claude-Code- und GitHub-Betriebssystems mit Repository-Truth-Model, Context Packs, Work Packages, kontinuierlichen Checkpoints, Handovers, Worktrees, PR-/CI-Gates, Agentenabbildung, Berechtigungen, Tests, Bauphasen und Startprompts	Erstellt